

DIRECTION NATIONALE DE L'ENSEIGNEMENT
SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE



Ecole Nationale d'Ingénieurs - Abderrahmane Baba Touré

410, Av. Van Vollenhoven BP 242 – Tél : (223) 20 22 27 36 – Fax : (223) 20 21 50 38 / Bamako
– MALI.

Département Génie Informatique & Télécommunications

**TP1 : Systèmes de Gestion de Bases de
Données (SGBD)**

Master GIT-S3

Matière	Administration des Bases de données	
Nom et Prénom du tuteur	Dr. Yacouba GOITA	
Noms et Prénoms	Drissa Sidiki Traore Pascal Diarra	
Date : 14/06/2025	Groupe : N°06	Durée :
Dépôt : 16/10/2025		

NOTE	APPRECIATION
..... /20	

ANNEE : 2024-2025

Table des matières

Objectif :	3
Structure des données :	3
Outils utilisés :	4
Partie 1 : Systèmes de Gestion de Bases de Données (SGBD)	4
Objectif :	4
Requêtes SQL réalisées :	4
Partie 2 : Data Control Language (DCL)	12
Objectif :	12
Requêtes SQL réalisées :	13
Conclusion :	14

Rapport du TP1

Objectif :

Ce TP a pour but de mettre en pratique les fondamentaux du langage SQL à travers deux volets : la manipulation des données relationnelles (requêtes SELECT, jointures, agrégats, etc.) et la gestion des droits d'accès via le DCL (création d'utilisateurs, rôles et attribution de privilèges).

Structure des données :

Le modèle relationnel portait sur une structure de gestion de cours et de formations, intégrant des entités comme :

- **Animateurs**
- **Cours**
- **Participants**
- **Thèmes**
- **Entreprises**
- **Catégories**
- Et les tables associatives **Animer** et **Participer**

Le Modèle Relationnel de Donnée (MRD) se présente comme suit :

Animateurs(matricule_animateur*(char(4), nom(varchar(30)), prénom(varchar(30)),)

Cours(code_cours*(char(4)), niveau(varchar(35), date_cours(date), tarif_hr(integer), prime_resp(integer), droit_inscrip(integer), code_theme(char(4)),, matricule_animateur(char(4)),)

Animer(code_cours*(char(4)), matricule_animateur*(char(4)), nbre_heures(integer)

Participants(matricule*(char(4)), nom(varchar(35), prénom(varchar(35), localité(varchar(35), code_entreprise (integer))

Catégories(code_catégorie*(char(4), libellé(varchar(35),)

Participer(code_cours*(char(4)), matricule) *(char(4))

Entreprises(code_entreprise*(char(4)), nom(varchar(35),), adresse(varchar(35),))

Thème(code_thème***(char(4)), désignation(varchar(35),), code_catégorie*(char(4))

Toutes les données de test, ainsi que la structure des tables utilisées dans ce TP, nous ont été fournies par le professeur afin d'assurer un cadre cohérent et standardisé pour les exercices.

Outils utilisés :

Pour la réalisation de ce TP, j'ai utilisé **XAMPP** accompagné de son interface **phpMyAdmin**, afin de gérer la base de données et exécuter les requêtes SQL dans un environnement local.



Partie 1 : Systèmes de Gestion de Bases de Données (SGBD)

Objectif :

Cette première partie du TP1 en Systèmes de Gestion de Bases de Données (SGBD) a pour objectif de **manipuler une base de données relationnelle en SQL** à travers des requêtes d'interrogation, de tri, de filtrage et d'agrégation, le tout basé sur un modèle relationnel bien défini.

Le TP s'inscrit dans une logique de mise en pratique des notions fondamentales de SQL : *SELECT, WHERE, GROUP BY, ORDER BY, JOIN, AVG, COUNT, HAVING*, etc.

Requêtes SQL réalisées :

1. **Requête 1** : Afficher le nom et le prénom de tous les animateurs.

```
SELECT Nom, Prenom FROM animateur;
```

Résultat :

Nom	Prenom
Goita	Yacouba
Dabo	Alpha
Camara	Lancine
Haidara	Balla
Kamissoko	Drissa

2. **Requête 2** : Afficher les différents droits d'inscription suivant l'ordre décroissant.

```
SELECT Droit_inscrip  
FROM cours  
ORDER BY `Droit_inscrip` DESC;
```

Résultat :

Droit_inscrip	1
	1000
	1000
	1000
	1000
	1000
	500
	500

3. **Requête 3 :** Afficher le code entreprise, les noms et prénoms des participants venant de l'entreprise dont le code est 3336.

```
SELECT `Code_entreprise`, `Nom`, `Prenom`  
FROM participants  
WHERE `Code_entreprise`=3336;
```

Résultat :

Code_entreprise	Nom	Prenom
3336	Diarra	Djeneba

4. **Requête 4 :** Afficher les matricules des animateurs responsables et la date des cours qui ont eu lieu entre le 03Avril et le 15 Avril 2022.

```
SELECT `Matricule_anim`, `Date_cours`  
FROM cours  
WHERE `Date_cours` BETWEEN '2022-04-03' AND '2022-04-15';
```

Résultat :

Matricule_anim	Date_cours
m002	2022-04-10
m002	2022-04-15

5. **Requête 5 :** Afficher les codes cours, le droit d'inscription et la prime de responsabilité des cours dont le droit d'inscription est en dessous de 6000 F et la prime de responsabilité est supérieure à 1000 F

```
SELECT `Code_cours`, `Droit_inscrip`, `Prime_resp`  
FROM cours WHERE `Droit_inscrip` < 6000 AND `Prime_resp` > 1000;
```

Résultat :

Code_cours	Droit_inscrip	Prime_resp
C003	1000	1500
C004	1000	1500
C005	1000	1500
C006	1000	1500
C007	1000	1500

6. **Requête 6 :** Afficher tous les noms et prénoms des participants dont le nom se termine avec la lettre N trié suivant l'ordre alphabétique des noms et prénoms.

```
SELECT `Nom`, `Prenom`
FROM participants WHERE `Nom` LIKE '%N'
ORDER BY `Nom`, `Prenom` ASC;
```

Résultat : Dans la source de donnée initiale, il n'existe aucun nom se terminant par N. Pour s'assurer que la requête fonctionne correctement, on a dû modifier le nom du participant p004 pour qu'il se termine par N.

Nom	Prenom
DiarraN	Djeneba

7. **Requête 7 :** Afficher tous les noms et prénoms des participants dont les 2e et 3e lettres du nom sont respectivement C et H.

```
SELECT `Nom`, `Prenom`
FROM `participants`
WHERE SUBSTRING(`Nom`, 2, 2) = 'ch';
```

Résultat : Dans la source de donnée initiale, il n'existe aucun nom dont son 2^e et 3^e lettre sont respectivement C et H, on a dû modifier les noms des participants p001 et p002 pour s'assurer que la requête fonctionne correctement.

Nom	Prenom
Dcho	Sekou
PCHI	Togobe

8. **Requête 8 :** Quel est le nombre de cours organisés ?

```
SELECT COUNT(*) AS Nb_cours FROM `cours`;
```

Résultat :

Nb_cours
7

9. **Requête 9 :** Calculer la moyenne des droits d'inscription, des primes de responsabilité et des tarifs heure.

```
SELECT
    AVG(`Droit_inscrip`) AS Moy_Droit_inscrip,
    AVG(`Prime_resp`) AS Moy_Prime_resp,
    AVG(`Tarif_hr`) AS Moy_Tarif_hr
FROM `cours`;
```

Résultat :

Moy_Droit_inscrip	Moy_Prime_resp	Moy_Tarif_hr
857.1429	1357.1429	8214.2857

10. **Requête 10** : Combien de cours organise-t-on pour chaque niveau ?

```
SELECT `Niveau`, COUNT(`Code_cours`) AS Nb_cours
FROM `cours`
GROUP BY `Niveau`;
```

Résultat :

Niveau	Nb_cours
a	2
d	2
i	3

11. **Requête 11** : Quel est le code du cours dont le droit d'inscription est le plus élevé ?

```
SELECT `Code_cours`, `Droit_inscrip`
FROM `cours` WHERE `Droit_inscrip` =
    (SELECT MAX(`Droit_inscrip`) FROM cours);
```

Résultat :

	Code_cours	Droit_inscrip
▶	C003	1000
	C004	1000
	C005	1000
	C006	1000
	C007	1000

12. **Requête 12** : Calculer le total des heures prestées par l'animateur ayant le numéro matricule 19800202222.

```
SELECT SUM(an.nbre_heures) AS Total_heures
FROM animateur a
JOIN animer an ON an.matricule_animateur = a.Matricule
WHERE a.Matricule = 'm001'
GROUP BY a.Matricule, a.Nom, a.Prenom;
```

Résultat : Il n'existe aucun animateur ayant le numéro matricule 19800202222. Pour s'assurer que la requête fonctionne correctement on a dû changer 19800202222 en un numéro matricule existant dans la table (m001).

Total_heures
60

13. **Requête 13 :** Afficher le numéro matricule et le nombre total d'heures prestées par chaque animateur trié suivant l'ordre croissant des numéros matricule.

```
SELECT a.Matricule, SUM(an.nbre_heures) AS Total_heures
FROM animateur a
JOIN animer an ON an.matricule_animateur = a.Matricule
GROUP BY a.Matricule
ORDER BY a.Matricule ASC;
```

Résultat :

Matricule	Total_heures
m001	60
m002	50
m003	20
m004	20
m005	30

14. **Requête 14 :** Afficher le numéro matricule et le nombre total d'heures prestées pour les animateurs ayant prestés plus de 10 heures.

```
SELECT a.Matricule, SUM(an.nbre_heures) AS Total_heures
FROM animateur a
JOIN animer an ON an.matricule_animateur = a.Matricule
GROUP BY a.Matricule
HAVING SUM(an.nbre_heures) > 10;
```

Résultat :

Matricule	Total_heures
m001	60
m002	50
m003	20
m004	20
m005	30

15. **Requête 15 :** Afficher le numéro matricule et le nombre total d'heures prestées pour les animateurs ayant prestés plus de 10 heures et qui sont nés en 68.

Cette requête tient compte d'un champ pour la date de naissance qui est inexistant.

Pour répondre correctement à la demande on a dû ajouter un champ **Date_naissance** et définir le champ Matricule comme étant la clé primaire avec les requêtes suivantes :

```
ALTER TABLE animateur ADD Date_naissance DATE;
ALTER TABLE animateur ADD PRIMARY KEY (Matricule);
On va maintenant insérer des données dans le champ
Date_naissance avec la requête suivante :
UPDATE animateur SET Date_naissance = '1968-03-
12' WHERE Matricule = 'm001';
UPDATE animateur SET Date_naissance = '1970-05-
23' WHERE Matricule = 'm002';
UPDATE animateur SET Date_naissance = '1968-11-
08' WHERE Matricule = 'm003';
UPDATE animateur SET Date_naissance = '1980-01-
15' WHERE Matricule = 'm004';
UPDATE animateur SET Date_naissance = '1968-07-
02' WHERE Matricule = 'm005'
```

Maintenant voici la requête :

```
SELECT a.Matricule, SUM(an.nbre_heures) AS Total_heures
FROM animateur a
JOIN animer an ON an.matricule_animateur = a.Matricule
WHERE YEAR(a.Date_naissance) = 1968
GROUP BY a.Matricule
HAVING SUM(an.nbre_heures) > 10;
```

Résultat :

Matricule	Total_heures
m001	60
m003	20
m005	30

16. **Requête 16** : Afficher pour chaque cours le code cours, le nom et le prénom de l'animateur responsable.

```
SELECT c.Code_cours, a.Nom, a.Prenom
FROM cours c
JOIN animateur a ON a.Matricule=c.Matricule_anim
GROUP BY c.Code_cours;
```

Résultat :

Code_cours	Nom	Prenom
C001	Dabo	Alpha
C002	Dabo	Alpha
C003	Goita	Yacouba
C004	Camara	Lancine
C005	Haidara	Balla
C006	Goita	Yacouba
C007	Kamissoko	Drissa

17. **Requête 17** : Afficher pour chaque cours son code, le thème et le nom et le prénom de l'animateur responsable.

```
SELECT c.Code_cours, t.Designation, a.Nom, a.Prenom
FROM cours c
JOIN animateur a ON a.Matricule = c.Matricule_anim
JOIN theme t ON t.Code_theme = c.Code_theme
GROUP BY c.Code_cours;
```

Résultat :

Code_cours	Designation	Nom	Prenom
C001	Prise en main du logiciel	Dabo	Alpha
C002	Travail pratique	Dabo	Alpha
C003	Concept avancé	Goita	Yacouba
C004	Travail pratique	Camara	Lancine
C005	Travail pratique	Haidara	Balla
C006	Conception	Goita	Yacouba
C007	Realisation	Kamissoko	Drissa

18. **Requête 18** : Pour chaque participant afficher le numéro matricule, le nom, le prénom ainsi que le nombre de cours auxquels il a participé.

```
SELECT p.Matricule_P, p.Nom, p.Prenom, COUNT(c.Code_cours) AS Nbre_cours
FROM participants p
LEFT JOIN participer pc ON pc.Matricule_P = p.Matricule_P
LEFT JOIN cours c ON c.Code_cours = pc.`Code-cours`
GROUP BY p.Matricule_P, p.Nom, p.Prenom;
```

Résultat :

Matricule_P	Nom	Prenom	Nbre_cours
p001	Dcho	Sekou	2
p002	PCHI	Togobe	1
p003	Ba	Therno	1
p004	DiarraN	Djeneba	0
p005	Konate	Solo	2
p006	Dembele	Moriba	2
p007	Tomota	Sira	1

19. **Requête 19** : Afficher le numéro matricule, le nom et le prénom ainsi que le nombre total d'heures prestées par chaque animateur.

```
SELECT
    a.Matricule,
    a.Nom, a.Prenom,
    SUM(an.nbre_heures) AS Total_heures
FROM animateur a
JOIN animer an ON an.matricule_animateur = a.Matricule
GROUP BY a.Matricule, a.Nom, a.Prenom;
```

Résultat :

Matricule	Nom	Prenom	Total_heures
m001	Goita	Yacouba	60
m002	Dabo	Alpha	50
m003	Camara	Lancine	20
m004	Haidara	Balla	20
m005	Kamissoko	Drissa	30

20. **Requête 20** : Afficher pour chaque entreprise (code et nom de l'entreprise) le nombre de participants trié suivant l'ordre croissant des noms d'entreprise.

```
SELECT
    e.Code_entreprise,
    e.Nom,
    COUNT(p.Matricule_P) AS Nbre_participants
FROM entreprise e
LEFT JOIN participants p ON p.Code_entreprise =
e.Code_entreprise
GROUP BY e.Code_entreprise, e.Nom
ORDER BY e.Nom ASC;
```

Résultat :

Code_entreprise	Nom ▲ 1	Nbre_participants
50010	Academie	1
03336	Bramali	1
40001	CSCOM	1
40000	M3	2
10005	Usine CMDT	2

21. **Requête 21** : Afficher le nom des catégories pour lesquelles il y a eu plus de 3 cours.

```
SELECT cat.Libelle
FROM categorie cat
JOIN theme t ON t.Code_entreprise = cat.Code_categorie
JOIN cours c ON c.Code_theme = t.Code_theme
GROUP BY cat.Libelle
HAVING COUNT(cat.Libelle) > 3;
```

Résultat :

Libelle
Informatique de gestion

22. **Requête 22** : Afficher les noms des animateurs qui ont animé plus de cours que l'animateur dont vous devez saisir le nom.

```
SELECT a.Nom
FROM animateur a
JOIN animer an ON an.matricule_animateur = a.Matricule
GROUP BY a.Nom
HAVING COUNT(a.Matricule) > (SELECT COUNT(a.Matricule)
FROM animateur a
JOIN animer an ON an.matricule_animateur = a.Matricule
WHERE a.Nom='Haidara');
```

Résultat :

Nom
Dabo
Goita

Partie 2 : Data Control Language (DCL)

Objectif :

Ce travail pratique a pour but de **manipuler le langage de contrôle des données (DCL)** en SQL. Il s'agissait de :

- Créer un utilisateur avec des contraintes de quota
- Créer un rôle
- Gérer des permissions (droits d'accès)
- Associer un rôle à un utilisateur
- Révoquer un droit spécifique

Toutes ces actions permettent de **sécuriser et contrôler l'accès aux données** dans un environnement multi-utilisateurs.

Requêtes SQL réalisées :

1. **Requête 1** : Créez un utilisateur nommé **etudiant1** avec un quota de 10 Mo dans le **tsp_compagnie**.

L'énoncé demande de créer un utilisateur et de lui attribuer un quota de **10 Mo** dans la base de données **tsp_compagnie**.

Cependant, il est important de souligner que ni **MySQL** ni **PostgreSQL** ne permettent de définir un quota de stockage (exprimé en octets ou en Mo) pour un utilisateur via une commande SQL.

Cette fonctionnalité est **propre à Oracle**, qui offre une gestion avancée des quotas à travers ses TABLESPACES, avec une syntaxe comme :

```
CREATE USER etudiant1 IDENTIFIED BY motdepasse
QUOTA 10M ON tsp_compagnie;
```

Dans notre contexte (MySQL ou PostgreSQL), cette commande **n'est pas applicable**, car ces SGBD **ne gèrent pas les quotas de taille au niveau SQL**. Les seules limitations possibles concernent :

- Le nombre de requêtes ou de connexions (dans MySQL)
- L'espace disque global via le système d'exploitation (dans PostgreSQL)

Ainsi, nous allons simplement créer l'utilisateur **etudiant1**, sans prendre en compte la contrainte de quota de 10 Mo.

```
CREATE USER 'etudiant1'@'localhost' IDENTIFIED BY '1234';
```

Résultat :

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,1833 seconde(s).)

```
CREATE USER 'etudiant1'@'localhost' IDENTIFIED BY '1234';
```

Cette commande crée un utilisateur nommé **etudiant1** avec le mot de passe **1234**

2. **Requête 2** : Créez un rôle nommé **r_etudiants**

```
CREATE ROLE 'r_etudiants';
```

Résultat :

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0342 seconde(s).)

```
CREATE ROLE 'r_etudiants';
```

3. **Requête 3** : Donnez les droits de sélection et de mise à jour sur la table Cours pour le rôle r_etudiants

```
GRANT SELECT, UPDATE ON cours TO 'r_etudiants';
```

Résultat :

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0577 seconde(s).)

```
GRANT SELECT, UPDATE ON tsp_compagnie.Cours TO 'r_etudiants';
```

4. **Requête 4** : Attribué le rôle r_etudiants à etudiant1

```
GRANT 'r_etudiants' TO 'etudiant1'@'localhost';
```

Résultat :

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,1988 seconde(s).)

```
GRANT 'r_etudiants' TO 'etudiant1'@'localhost';
```

5. **Requête 5** : Enlevez le droit de mise à jour sur le rôle r_etudiants pour la table Cours

```
REVOKE UPDATE ON cours FROM 'r_etudiants';
```

Résultat :

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0302 seconde(s).)

```
REVOKE UPDATE ON tsp_compagnie.Cours FROM 'r_etudiants';
```

Conclusion :

Ce TP nous a permis d'explorer de manière concrète deux dimensions essentielles de la gestion de bases de données relationnelles :

d'une part, la **manipulation des données** via des requêtes SQL précises et variées, et d'autre part, la **gestion des utilisateurs et des privilèges** grâce au DCL.

À travers ces deux volets, nous avons pu non seulement renforcer notre maîtrise du langage SQL, mais aussi comprendre l'importance de la **sécurité, du contrôle d'accès** et de la **structuration logique des droits** dans un système multi-utilisateur.

Ce travail pratique nous a ainsi offert une vision complète et équilibrée des aspects **fonctionnels** et **organisationnels** d'un système de gestion de base de données.